Laporan Tugas Besar 1
IF2211 - Strategi Algoritma
Pemanfaatan Algoritma Greedy dalam pembuatan bot Permainan Battlecode 2025

Disusun oleh:

Kelompok : Probe
Muhammad Adam Mirza/18223015
Nazwan Siddqi Muttaqin/18223066
Matthew Sebastian Kurniawan/18223096

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

DAFTAR ISI

I. Deskripsi Tugas.....3
II. Landasan Teori.....4
2.1. Dasar Teori.....4
2.2. Cara Kerja Program Secara Umum.....4
III. Aplikasi Strategi Greedy..... 7
3.1. Mapping Persoalan.....7
3.2. Eksplorasi 3 Alternatif..... 8
1. Alternatif 1..... 8
2. Alternatif 2..... 10
3. Alternatif 3..... 12
3.3. Analisis Efisiensi dan Efektivitas..... 13
1. Alternatif 1..... 13
2. Alternatif 2..... 14
3. Alternatif 3..... 14
3.4. Pemilihan Strategi Greedy..... 14
IV. Implementasi dan Pengujian..... 16
4.1. Implementasi Solusi.....16
1. Alternatif 1 (Mainbot).....16
2. Alternatif 2.....21
3. Alternatif 3.....24
4.2. Penjelasan Struktur Data, Fungsi, dan Prosedur.....27
1. Run.....27
2. RunTower.....27
3. RunSoldier..... 28
4. RunSplasher..... 29
5. RunMopper..... 30
6. ProcessMessages.....31
7. RobotReport.....31
8. Score Functions (heuristics).....32
4.3. Pengujian Greedy.....33
1. Pengujian 1 (Main Bot vs Alternative Bot 1) – AlarmClock.....33
2. Pengujian 2 (Main Bot vs Alternative Bot 1) – Leaf..... 34
3. Pengujian 3 (Main Bot vs Alternative Bot 2) – AlarmClock.....34
4. Pengujian 4 (Main Bot vs Alternative Bot 2) – UnderTheSea..... 35
4.4. Analisis Hasil Pengujian.....35
1. Pengujian 1 (Main Bot vs Alternative Bot 1) – AlarmClock.....35

2. Pengujian 2 (Main Bot vs Alternative Bot 1) – Leaf..... 35
3. Pengujian 3 (Main Bot vs Alternative Bot 2) – AlarmClock.....36
4. Pengujian 4 (Main Bot vs Alternative Bot 2) – UnderTheSea..... 36
V. Kesimpulan dan Saran.....37
5.1. Kesimpulan.....37
5.2. Saran..... 37
VI. Lampiran.....38
VII. Daftar Pustaka..... 39

I. Deskripsi Tugas

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

II. Landasan Teori

2.1. Dasar Teori

Algoritma *Greedy* merupakan salah satu metode penyelesaian masalah yang paling sederhana dan populer, khususnya untuk memecahkan persoalan optimasi (baik maksimasi maupun minimasi). Prinsip utama dari algoritma ini adalah "mengambil apa yang bisa didapatkan sekarang". Artinya, pada setiap langkah penyelesaian, algoritma *greedy* akan selalu membuat pilihan lokal yang tampak paling menguntungkan atau optimal pada saat itu (*local optimum*), tanpa memikirkan konsekuensi jangka panjangnya. Harapannya, kumpulan pilihan optimal lokal tersebut akan mengarah pada solusi optimal secara global (*global optimum*).

Dalam penerapannya, algoritma *greedy* dibentuk oleh lima elemen utama, yaitu:

1. **Himpunan Kandidat:** Kumpulan elemen atau pilihan yang tersedia pada setiap langkah untuk membentuk solusi.
2. **Himpunan Solusi:** Kumpulan kandidat yang telah terpilih sebagai bagian dari penyelesaian masalah.
3. **Fungsi solusi:** Berfungsi untuk menentukan apakah solusi yang terbentuk sudah lengkap atau belum.
4. **Fungsi Seleksi (*Selection Function*):** Fungsi heuristik yang digunakan untuk menilai dan memilih kandidat terbaik dari himpunan kandidat yang layak.
5. **Fungsi Kelayakan (*Feasible Function*):** Fungsi yang bertugas memeriksa apakah suatu kandidat dapat dimasukkan ke dalam himpunan solusi tanpa melanggar aturan atau batasan masalah yang ada.
6. **Fungsi Objektif (*Objective Function*):** Nilai atau tujuan akhir yang ingin dimaksimalkan (profit/keuntungan) atau diminimalkan (biaya/waktu) oleh algoritma.

2.2. Cara Kerja Program Secara Umum

2.2.1. Cara Kerja dan Aksi Bot

Permainan Battlecode berjalan dengan sistem *turn-based* (berbasis giliran). Pada setiap rondenya, seluruh robot yang hidup akan dieksekusi kodenya secara bergantian. Karena sumber daya komputasi sangat dibatasi menggunakan sistem hitungan *bytecode*, bot harus mengeksekusi logika dan aksinya seefisien mungkin.

Secara umum, alur hidup bot pada setiap giliran adalah sebagai berikut:

1. **Sensing & Observasi:** Bot memindai sekelilingnya menggunakan radius pandang yang dimiliki untuk mendapatkan informasi petak (*MapInfo*), posisi musuh, posisi

tower kawan, serta keberadaan reruntuhan (*ruins*). Bot juga akan membaca pesan masuk (*messages*) dari bot sekutu untuk memperbarui strategi.

2. **Evaluasi Keadaan:** Bot mengevaluasi status dirinya, seperti jumlah cat (*paint*) yang tersisa, *health points* (HP), serta apakah *action cooldown* dan *movement cooldown* memungkinkan untuk melakukan aksi pada *turn* tersebut.
3. **Eksekusi Aksi:** Berdasarkan prioritas, bot melakukan tindakan (bergerak, mewarnai petak, menghapus cat musuh, membangun *tower*, atau mentransfer cat).
4. **Yielding:** Setelah komputasi dan aksi selesai, bot akan memanggil fungsi *Clock.yield()* untuk dengan sengaja mengakhiri gilirannya. Hal ini krusial untuk mencegah terpotongnya eksekusi di tengah jalan akibat kehabisan *bytecode* yang dapat menyebabkan perilaku tidak normal.

2.2.2. Implementasi Algoritma Greedy pada Bot

Secara umum, implementasi algoritma *greedy* pada bot ini bertumpu pada evaluasi keuntungan sesaat (*immediate reward*) pada satu giliran. Daripada menggunakan algoritma penelusuran graf atau pencarian jalur yang mendalam (seperti *A-Star* atau *Minimax*) yang memakan banyak *bytecode*, bot ini dirancang untuk menjadi sangat *greedy* dan reaktif.

Ide utamanya adalah menggunakan sistem **Penilaian Heuristik (*Heuristic Scoring*)**. Setiap kali sebuah robot memiliki kesempatan untuk bergerak atau melakukan aksi, program akan mendaftar semua opsi legal yang bisa dilakukan saat itu juga. Setiap opsi kemudian dievaluasi oleh fungsi heuristik yang akan memberikan nilai (skor). Aksi yang memberikan keuntungan langsung terbesar (misalnya menyelesaikan pembangunan menara, menghancurkan musuh, atau membersihkan cat musuh) akan diberi skor tertinggi, sedangkan aksi yang merugikan (seperti melangkah ke cat musuh) akan diberi skor penalti. Bot kemudian akan mengeksekusi opsi dengan skor tertinggi tersebut tanpa memikirkan rencana jangka panjang di ronde-ronde berikutnya.

2.2.3. Cara Menjalankan Bot

Untuk menjalankan dan mensimulasikan bot di dalam permainan, langkah-langkah yang dilakukan adalah sebagai berikut:

1. Pastikan *source code* bot sudah berada di dalam folder src/ (misalnya pada direktori src/mainBot/).
2. Buka terminal pada *root directory* dari proyek (direktori utama).
3. Jalankan perintah kompilasi menggunakan Gradle dengan mengetikkan ./gradlew build.
4. Setelah *build* berhasil tanpa *error*, buka folder client dan jalankan aplikasi *client* Battlecode (berupa file *executable* yang telah disediakan).

5. Pada antarmuka *client*, atur *directory root* ke folder utama proyek (bukan folder src).
6. Pilih bot yang akan ditandingkan (misalnya mainBot melawan tim lawan atau alternativeBots1), lalu jalankan simulasi (*queue game*) untuk melihat visualisasi pertandingan.

III. Aplikasi Strategi *Greedy*

3.1. Mapping Persoalan

Permainan Battlecode 2025 mengharuskan pemain mengendalikan pasukan robot secara otonom dalam lingkungan berbasis giliran (*turn-based*). Tujuan utama (obyektif global) adalah mewarnai lebih dari 70% petak di peta atau menghancurkan seluruh unit musuh. Karena batasan sumber daya komputasi (*bytecode*) yang sangat ketat, setiap robot harus mengambil keputusan secara cepat dan mandiri pada setiap gilirannya.

Persoalan ini dipetakan ke dalam enam elemen algoritma *greedy* dengan sudut pandang optimasi aksi per giliran (*per-turn action optimization*):

1. Himpunan Kandidat (C)

   Himpunan kandidat berisi seluruh kemungkinan aksi (pergerakan dan kemampuan khusus) yang dapat dilakukan oleh sebuah unit (robot maupun menara) pada satu giliran tertentu. Dalam konteks spesifikasi Battlecode, himpunan kandidat ini mencakup:
   - Aksi pergerakan: Bergerak ke salah satu dari 8 petak tetangga yang valid.
   - Aksi utama (*action*): Menyerang/mewarnai petak, menghapus cat musuh (*mop*), melakukan *mop swing*, menyelesaikan pola *tower* (completeTowerPattern), menandai pola (markTowerPattern), mentransfer cat, menarik cat dari menara (*withdraw*), atau membangun robot (khusus menara).

2. Himpunan Solusi (S)

   Himpunan solusi merupakan kandidat aksi terpilih yang pada akhirnya benar-benar dieksekusi oleh unit pada giliran tersebut. Karena sistem permainan memisahkan waktu tunggu (*cooldown*) antara pergerakan dan aksi utama, himpunan solusi untuk satu robot dalam satu giliran bisa berisi maksimal satu aksi pergerakan dan satu aksi utama.

3. Fungsi Kelayakan (*Feasible Function)*

   Fungsi kelayakan adalah sekumpulan aturan validasi dari API Battlecode yang memastikan bahwa kandidat aksi tidak melanggar batasan permainan. Sebuah kandidat aksi dinyatakan layak masuk ke dalam proses seleksi jika memenuhi kondisi berikut:

   - Pengecekan *cooldown*: rc.isMovementReady() bernilai *true* (waktu tunggu pergerakan < 10) atau rc.isActionReady() bernilai *true* (waktu tunggu aksi < 10).
   - Pengecekan legalitas target: rc.canMove(direction), rc.canAttack(location), rc.canTransferPaint(), atau rc.canCompleteTowerPattern().

- Pengecekan sumber daya: Robot memiliki cat (*paint*) dan tim memiliki uang (*chips*) yang cukup untuk mengeksekusi aksi tersebut.

4. Fungsi Seleksi *(Selection Function)*

Fungsi seleksi merupakan heuristik *greedy* utama yang bertugas memberikan bobot atau skor (score) pada setiap kandidat aksi yang lolos fungsi kelayakan. Pembobotan ini ditentukan berdasarkan seberapa menguntungkan aksi tersebut pada kondisi saat itu (diimplementasikan di dalam modul Heuristics.java). Contohnya:

- Menyelesaikan pembangunan *Money Tower* diberikan bobot sangat tinggi (SCORE_COMPLETE_MONEY_TOWER = 10000).
- Bagi robot *Mopper*, membersihkan cat musuh diberikan skor SCORE_MOP_ENEMY_PAINT = 800.
- Bergerak ke petak yang berisi cat musuh akan diberikan penalti (PENALTY_ENEMY_PAINT = -500) untuk *Soldier*, namun justru diberikan bonus untuk *Mopper*. Algoritma akan selalu membandingkan skor ini (mencari bestScore) dan memilih aksi dengan skor paling maksimum secara rakus.

5. Fungsi Solusi

Fungsi solusi bertugas menentukan kapan proses pencarian aksi pada satu giliran dianggap selesai. Hal ini terpenuhi ketika robot telah mengeksekusi kandidat aksi terbaiknya dan tidak ada lagi *cooldown* yang tersisa, atau batas evaluasi telah tercapai. Pada program, hal ini ditandai dengan berakhirnya blok iterasi penentuan keputusan, lalu robot memanggil perintah Clock.yield() untuk menyerahkan kembali kontrol kepada *game engine* dan melanjutkan ke giliran robot selanjutnya.

6. Fungsi Objektif

- **Objektif Lokal (Fokus Greedy):** Memaksimalkan akumulasi skor heuristik pada setiap giliran yang berjalan.
- **Objektif Global:** Mewarnai petak lebih banyak dari lawan (dominasi wilayah) dan mengamankan suplai *paint* serta *chips* demi memenangkan permainan.

3.2. Eksplorasi 3 Alternatif

1. Alternatif 1

Pada alternatif pertama ini, dirancang sebuah strategi *greedy* yang berfokus pada agresivitas ekspansi, dominasi teritorial, dan spesialisasi peran. Algoritma *greedy* pada bot ini tidak melakukan perencanaan jangka panjang, melainkan secara rakus menghitung

skor dari setiap probabilitas aksi yang valid di satu giliran, lalu mengeksekusi aksi dengan perolehan poin tertinggi (bestScore).

Berikut adalah rincian fungsionalitas dan pembobotan heuristik pada alternatif pertama:

- **Prioritas Mutlak Pembangunan Menara (*Tower Building*)** Objektif penguasaan peta sangat bergantung pada keberadaan menara sebagai titik *spawn* dan penyuplai sumber daya. Oleh karena itu, fungsi heuristik memberikan nilai ekstrem untuk aksi yang berkaitan dengan penyelesaian menara:
  - Aksi completeTowerPattern untuk *Money Tower* diberikan bobot tertinggi di seluruh permainan, yaitu 10.000 poin, disusul oleh *Paint Tower* sebesar 8.000 poin, dan *Defense Tower* sebesar 6.000 poin.
  - Jika sebuah robot (*Soldier*) mendeteksi reruntuhan (*ruin*) dan memiliki kemampuan untuk menyelesaikannya, bot akan secara rakus mengabaikan semua opsi lain (termasuk menyerang atau berpindah) demi membangun menara tersebut.

- **Spesialisasi Heuristik Berdasarkan Tipe Robot** Setiap tipe robot memiliki fungsi objektif lokalnya masing-masing yang diatur melalui pembobotan skor pergerakan dan serangan:
  - **Soldier:** *Soldier* bertugas murni untuk mewarnai peta dan mencari reruntuhan. Fungsi heuristik pergerakannya (scoreSoldierMove) memberikan bonus besar (+200 poin) jika robot bergerak mendekati *ruin* terdekat. Sebaliknya, untuk meminimalisasi kerugian, *Soldier* diberikan penalti (-500 poin) jika mengevaluasi pergerakan ke petak yang berisi cat musuh, sehingga ia akan secara *greedy* menghindarinya.
  - **Splasher:** *Splasher* dirancang untuk melakukan *rush* ke markas atau menara musuh. Heuristik pergerakannya (scoreSplasherMove) memberikan bonus +500 poin jika bergerak mendekati target musuh. Jika tidak ada target intelijen, bot akan secara *greedy* mengekspansi area dengan bergerak ke tengah peta (bonus +200) serta bergerak menjauhi menara kawan (bonus +150 jika radius kuadrat <= 25) agar persebaran pasukan lebih luas. Untuk aksi serangan, menyiram cat di dekat menara musuh diberikan bonus destruktif sebesar +500 poin.
  - **Mopper:** Pergerakan *Mopper* (scoreMopperMove) berkebalikan dengan *Soldier*, ia ditarik secara *greedy* oleh keberadaan cat musuh (+200 poin). Aksi membersihkan cat musuh (*mop*) diberikan skor dasar tinggi (800 poin) dikurangi jarak robot ke target. Apabila mendeteksi gerombolan musuh, heuristik *mop swing* mendominasi dengan skor 400 ditambah 200 poin untuk setiap musuh yang ada di area jangkauan.

- **Manajemen *Spawn* Unit pada Menara,** keputusan menara untuk memproduksi (*spawn*) robot baru tidak dilakukan secara acak, melainkan menggunakan fungsi heuristik yang dinamis bergantung pada fase permainan (*early* vs *late game*) dan putaran *turn* (ronde) saat itu. Contohnya, fungsi *greedy* pada menara akan sangat rakus melakukan *spawn Soldier* di awal permainan (diberikan skor 800) untuk mempercepat ekspansi, namun menggeser prioritasnya untuk memproduksi *Splasher* (skor 900) di akhir permainan guna melakukan agresi ke markas lawan. Selain itu, terdapat logika periodik yang memprioritaskan *spawn Mopper* (skor 2000) atau *Soldier* tambahan pada putaran-putaran tertentu untuk menjaga keseimbangan komposisi pasukan.

- **Komunikasi Berbasis *Greedy*,** keputusan *greedy* robot sangat bergantung pada informasi lokasi strategis. Diimplementasikan sebuah sistem komunikasi (Comms.java) di mana menara dan robot saling berbagi koordinat musuh dan reruntuhan. Informasi ini secara langsung mempengaruhi nilai heuristik robot. Misalnya, jika pesan menyatakan adanya menara musuh, target *rush* dari robot *Splasher* akan langsung beralih menuju koordinat tersebut untuk dihancurkan.

2. Alternatif 2

Pada alternatif kedua ini, dirancang sebuah strategi *greedy* yang berfokus pada manajemen tugas berbasis *stack* (StackManager), respons garis depan (battlefront), dan siklus hidup menara yang agresif. Algoritma pada bot ini secara greedy mengevaluasi lingkungan lokalnya pada satu giliran untuk menentukan aksi dengan perolehan utilitas tertinggi, sekaligus menggunakan struktur *stack* untuk memastikan fungsi kelayakan prioritas utama tidak mudah terdistraksi.

Berikut adalah rincian fungsionalitas dan pembobotan heuristik pada alternatif kedua:

- **Fokus Utama pada Pembangunan Menara (Tower Building)** secara *greedy*, fungsi seleksi mengevaluasi himpunan kandidat reruntuhan yang terlihat di peta untuk dikuasai dan ditambahkan ke dalam tumpukan solusi (Task.BUILD_RUIN). Heuristiknya menghitung nilai secara *greedy* dengan memprioritaskan jarak terpendek (meminimalkan distanceSquaredTo). Berbeda dengan alternatif lain yang adaptif, fungsi objektif pada bot ini memiliki ketetapan statis yang sangat *greedy* terhadap ekonomi. Fungsi seleksi secara *greedy* hanya akan memilih kandidat pembangunan *Money Tower* (100%) di sepanjang permainan tanpa adanya pergeseran fase ke *Paint Tower*. Selain itu, fungsi kelayakan pada tahap ini sangat ketat, jika terdeteksi musuh dalam radius 20 petak dari reruntuhan, robot akan secara *greedy* membatalkan pembangunan dan beralih menyerang musuh terlebih dahulu.

- **Spesialisasi Heuristik Berdasarkan Tipe Robot,** setiap tipe robot memiliki fungsi kelayakan dan fungsi objektif tempur yang unik:
    - **Soldier:** *Soldier* bertindak sebagai robot pembangun dan penjaga. Untuk pemilihan target serang, fungsi objektifnya mengevaluasi himpunan musuh dan secara *greedy* memprioritaskan musuh tipe *mobile* yang memiliki HP (darah) paling rendah untuk segera dieliminasi dari peta.
    - **Splasher:** Fungsi seleksinya *Splasher* mengevaluasi seluruh petak di sekitarnya sebagai himpunan kandidat. Fungsi objektif memberikan bobot statis, +3 poin jika berisi cat musuh, +1 poin jika kosong, dan 0 jika cat kawan. Splasher secara *greedy* akan menembakkan cat ke koordinat dengan akumulasi skor tertinggi demi efisiensi pemakaian cat.
    - **Mopper:** *Mopper* Fokus pada pembersihan lokal. Fungsi seleksi mengevaluasi musuh *mobile* di sekitarnya dan meminimalkan jarak (bestDist) sebagai nilai fungsi objektifnya. Himpunan kandidat juga mencakup petak berisi cat musuh. Mopper secara *greedy* akan membersihkan cat musuh terdekat untuk segera merusak jalur pergerakan lawan.

- **Fungsi Kelayakan Logistik (Refill Threshold),** setiap robot dilengkapi dengan batasan kelayakan (feasibility). Apabila level cat berada di bawah ambang batas (contohnya, < 20% pada Splasher/Mopper, atau < 25% pada Soldier), fungsi seleksi otomatis menolak aksi tempur. Robot akan secara *greedy* memasukkan tugas REFILL ke dalam *stack* untuk segera menyedot cat cadangan dari menara kawan terdekat.

- **Siklus Hidup Menara Agresif,** menara menggunakan fungsi objektif berdasar urutan statis, secara greedy memproduksi 2 *Soldier* pertama, lalu prioritas digeser ke *Mopper* jika mendeteksi ancaman musuh, atau *Splasher* jika ekonomi *chips* sangat berlebih (>= 1200). Uniknya, terdapat heuristik siklus hidup untuk efisiensi. Jika, menara sekunder usianya mencapai 50 giliran, fungsi seleksi akan memilih kandidat aksi hancur diri (*disintegrate*) secara *greedy* untuk mendaur ulang resource dan mencegah musuh mengambil keuntungan dari menara tersebut.

- **Komunikasi Garis Depan (*Battlefront*)**, titik kontak pertama dengan musuh (MSG_BATTLEFRONT) bertindak sebagai target fungsi objektif pemandu. Pesan ini direlai oleh menara ke unit-unit baru, sehingga robot yang tidak memiliki tujuan spesifik akan secara *greedy* menggunakan navigasi BugNav untuk berbondong-bondong menuju titik pusat pertempuran tersebut.

3. Alternatif 3

Pada alternatif ketiga ini, dirancang sebuah strategi *greedy* yang berfokus pada prioritas objektif global dan respons terhadap sinyal komunikasi. Algoritma *greedy* pada bot ini mengevaluasi berbagai target (*goal*) yang ada di peta sebagai himpunan kandidat, menghitung poin prioritas dan skor utilitasnya di setiap giliran, lalu secara *greedy* mengeksekusi aksi dengan prioritas (bestPriority) dan perolehan poin tertinggi (bestScore). Berbeda dengan alternatif sebelumnya, fungsi heuristik pada bot ini sangat dipengaruhi oleh fase permainan dan informasi dari robot lain.

Berikut adalah rincian fungsionalitas dan pembobotan heuristik pada alternatif ketiga:

- **Fokus Utama pada Pembangunan Menara (*Tower Building*)** secara *greedy*, fungsi seleksi (GoalManager) memberikan bobot ekstrem untuk himpunan kandidat aksi yang berkaitan dengan penguasaan reruntuhan (GOAL_CAPTURE_RUIN). Heuristiknya menghitung nilai secara *greedy* dengan rumus dasar 10.000 dibagi jarak. Terdapat pergeseran heuristik secara dinamis, di fase awal, fungsi objektif memberikan skor penuh (100%) hanya untuk membangun Money Tower. Namun, saat memasuki fase pertengahan dan akhir, fungsi seleksi secara greedy hanya akan memilih kandidat pembangunan *Paint Tower* demi dominasi cat.

- **Spesialisasi Heuristik Berdasarkan Tipe Robot** Setiap tipe robot memiliki fungsi objektif lokalnya masing-masing yang diatur melalui pembobotan pergerakan, tugas, dan manajemen cat:
    - **Soldier:** *Soldier* bertindak mengevaluasi opsi pergerakan. Jika, mendeteksi reruntuhan yang dijaga musuh, nilai heuristik GOAL_CONTEST_RUIN mendominasi sehingga fungsi seleksi secara *greedy* memilih untuk merebut area tersebut. Jika tidak ada reruntuhan, Soldier secara greedy mencari dan menargetkan musuh terdekat di dalam jangkauannya.
    - **Splasher:** Fungsi seleksi *Splasher* (selectSplasherTarget) tidak menembak secara acak, melainkan menghitung himpunan solusi area 5x5 di sekitarnya. Splasher memberikan bonus +2 poin untuk setiap petak cat musuh dan +1 poin untuk petak kosong. Algoritma akan secara greedy mengeksekusi serangan pada titik pusat yang memberikan total bestScore destruktif tertinggi. Jika, cat menipis (<100), fungsi heuristik akan membatalkan semua prioritas serang dan secara *greedy* menarik robot kembali ke menara terdekat untuk *refill*.

- **Mopper:** Fungsi seleksi ayunan *Mopper* (selectMopperSwingDirection) mengevaluasi ke-8 arah mata angin sebagai kandidat, menghitung berapa banyak musuh di dalam jangkauan ayunan, lalu secara greedy memilih arah dengan total '*hits*' terbanyak. Selain itu, jika fungsi evaluasi mendeteksi *Soldier* kawan yang kritis cat (< 50) di dekatnya, heuristik distribusi sumber daya akan memberikan bobot prioritas tertinggi pada aksi transfer cat. Sehingga, fungsi seleksi secara *greedy* memilih untuk membagikan cat dari persediaannya agar ekspansi tim tidak terhenti.

- **Manajemen Spawn Unit pada Menara**, Keputusan menara memproduksi unit baru tidak acak, melainkan menggunakan fungsi seleksi probabilitas statis yang menyesuaikan fase ronde permainan (roundNum). Di awal permainan, fungsi seleksi *greedy* memilih spawn Soldier (probabilitas 80%). Di pertengahan permainan, rasio pemilihan Splasher dinaikkan (20%). Di fase akhir, komposisi solusi digeser secara agresif menjadi 60% Soldier, 30% Splasher, dan 10% Mopper.

- **Komunikasi Berbasis Sinyal**, Pesan dari sistem komunikasi jaringan bertindak sebagai pengubah nilai fungsi objektif. Jika, sebuah robot memancarkan sinyal darurat (MSG_ENEMY_TOWER atau MSG_BATTLEFRONT), pesan ini dievaluasi dengan bobot sangat besar (20.000 / jarak). Fungsi seleksi robot yang menerima sinyal ini akan membatalkan himpunan solusi sebelumnya dan secara *greedy* memilih untuk melakukan rush menuju koordinat tersebut.

3.3. Analisis Efisiensi dan Efektivitas

Analisis terhadap ketiga alternatif solusi *greedy* dilakukan berdasarkan dua metrik utama, yaitu Efisiensi komputasi dan Efektivitas heuristik. Meskipun ketiganya berbasis pendekatan *greedy*, terdapat perbedaan performa:

1. Alternatif 1
   - Efisiensi: Merupakan algoritma yang paling ringan dan cepat. Kompleksitas waktu eksekusinya mutlak O(1) di setiap giliran karena fungsi seleksinya hanya mengevaluasi himpunan kandidat dalam radius statis (8 arah mata angin atau 9 petak terdekat). Selain itu, jejak memorinya sangat minimalis (kompleksitas ruang O(1)) karena *bot* ini tidak menyimpan matriks peta sama sekali.
   - Efektivitas: Sangat menonjol dalam hal *mikro-manajemen unit* dan efisiensi pengeluaran sumber daya. Namun, pergerakannya yang murni berdasar *greedy* statis tanpa algoritma pathfinding membuatnya rentan mengalami kebuntuan di jalur sempit (*chokepoint*).

2. Alternatif 2
    - Efisiensi: Sangat hemat memori (kompleksitas ruang O(1)) karena manajemen tugasnya hanya diatur oleh antrean *Stack* berukuran kecil. Namun, kompleksitas waktu komputasinya (tetap O(1) per langkah) sedikit lebih tinggi dibandingkan Alternatif 1 karena fungsi navigasinya (BugNav) menyertakan iterasi konstan (hingga 24 kali putaran) saat harus menelusuri rintangan.
    - Efektivitas: Eksekusi pembangunan *Money Tower* tanpa henti menjadikan bot ini sangat cepat menguasai ekonomi di awal permainan. Tetapi, fungsi objektifnya terlalu *myopic*. Karena secara *greedy* murni selalu mengejar target musuh terdekat di dalam radiusnya, bot ini sangat mudah dijebak oleh musuh.

3. Alternatif 3
    - Efisiensi: Merupakan algoritma yang paling berat secara komputasi (kurang efisien). Membutuhkan kompleksitas ruang setidaknya O(N) untuk menyimpan pembaruan matriks MapData secara persisten sesuai luas arena. Selain itu, fungsi pencarian targetnya memakan waktu O(v+k) untuk memindai array sisa reruntuhan dan objek-objek baru.
    - Efektivitas: Meskipun memakan lebih banyak memori, ini adalah strategi penyelesaian masalah yang paling unggul. Kelemahan rabun dekat pada algoritma *greedy* berhasil diatasi menggunakan sistem pesan komunikasi. Dengan menjadikan sinyal markas musuh sebagai nilai fungsi objektif paling ekstrem (20.000 poin), keputusan *greedy* di tingkat lokal berhasil disinkronkan menjadi serangan massal di tingkat global. Pergeseran fungsi seleksi secara dinamis dari *Money Tower* ke *Paint Tower* juga memastikan bot selalu relevan hingga fase *late game*.

3.4. Pemilihan Strategi *Greedy*

Berdasarkan hasil analisis perbandingan antara ketiga alternatif solusi greedy, strategi yang dipilih untuk diimplementasikan menjadi main bot adalah **Alternatif 1**.

Pemilihan ini didasarkan pada pertimbangan mengenai keseimbangan optimal antara efisiensi komputasi dan fleksibilitas *mikro-manajemen unit* di arena pertempuran. Alternatif 1 memiliki kompleksitas waktu dan ruang mutlak O(1) di setiap giliran, sehingga tidak membebani memori dengan pemetaan matriks persisten seperti pada Alternatif 3 ataupun iterasi penelusuran rintangan yang berulang pada Alternatif 2. Selain unggul secara algoritmik, fungsi objektif Alternatif 1 beroperasi menggunakan sistem skor terpadu yang mengevaluasi seluruh probabilitas opsi secara paralel, memungkinkan robot secara greedy mengeksekusi aksi sekunder (aksi bonus) dan melakukan manajemen logistik cat secara

mandiri dengan sangat presisi. Meskipun tidak dilengkapi dengan sistem koordinasi armada berskala global yang masif seperti Alternatif 3, kecepatan komputasinya yang baik, dikombinasikan dengan heuristik spesialisasi peran yang reaktif dan agresif dalam berekspansi, menjadikan Alternatif 1 sebagai solusi yang paling konsisten dan sangat aman terhadap batasan komputasi (Time/Bytecode Limit) untuk menyelesaikan persoalan pada permainan ini.

IV. Implementasi dan Pengujian

4.1. Implementasi Solusi

1. Alternatif 1 (*Mainbot*)

```
BEGIN

  // Inisialisasi global
  turnCount <- 0
  spawnLoc <- GetLocation()

  possibleBases[0] <- Rotate180(spawnLoc)
  possibleBases[1] <- ReflectHorizontal(spawnLoc)
  possibleBases[2] <- ReflectVertical(spawnLoc)

  enemyBase <- possibleBases[0]
  currentBaseIndex <- 0


  WHILE IsRunning DO

    turnCount <- turnCount + 1
    unitType <- GetType()


    IF unitType = TOWER THEN

      ProcessMessages()
      TowerAttackLowestHP()

      // Evaluasi kandidat spawn secara greedy
      bestScore <- -INF
      bestUnit <- NULL

      FOR each u IN {SOLDIER, SPLASHER, MOPPER} DO
        score <- ScoreTowerSpawn(u, round) + Random(0, 29)

        IF score > bestScore THEN
          bestScore <- score
          bestUnit <- u
```

```
      ENDIF
    ENDFOR


    // Cari posisi spawn valid pertama
    bestDir <- FirstBuildableDirection(bestUnit)

    IF bestUnit != NULL AND bestDir != NULL THEN
      BuildRobot(bestUnit, bestDir)
    ENDIF


    TowerRelayAndCommand()

  ENDIF



  IF unitType = SOLDIER THEN

    ProcessMessages()

    nearestRuin <- FindNearestEmptyRuin()
    nearestTower <- FindNearestAllyTower()

    IF nearestRuin = NULL AND HasRecentRuinIntel() THEN
      nearestRuin <- lastRuinFromIntel
    ENDIF


    bestAction <- ACT_NOTHING
    bestScore <- -INF


    // Kandidat 1: complete tower
    EvaluateCompleteTowerPattern()

    // Kandidat 2: paint pattern tile yang salah
    EvaluatePaintPatternTile()
```

```
      // Kandidat 3: mark tower pattern
      EvaluateMarkTowerPattern()

      // Kandidat 4: resource pattern (jika tidak ada
ruin)
      EvaluateResourcePatternIfNoRuin()

      // Kandidat 5: withdraw paint saat paint rendah
      EvaluateWithdrawPaintIfLow(nearestTower)

      // Kandidat 6: upgrade ally tower
      EvaluateUpgradeTower()

      // Kandidat 7: move 8 arah
      EvaluateMoveAllDirections(nearestRuin, nearestTower,
attackOrder)


      Execute(bestAction)


      // Bonus movement
      IF bestAction != ACT_MOVE AND bestAction !=
ACT_NOTHING THEN
        IF IsMovementReady() AND nearestRuin != NULL THEN
          MoveTo(nearestRuin)
        ENDIF
      ENDIF


      // Bonus action prioritas:
      // 1) complete tower siap jadi
      // 2) paint pattern tile salah
      // 3) paint tile sekitar yang non-ally
      BonusFollowupActionSoldier()


      RobotReport(nearestRuin)

    ENDIF
```

```
    IF unitType = SPLASHER THEN

      ProcessMessages()

      nearestTower <- FindNearestAllyTower()
      rushTarget <- SelectRushTarget(enemyTowerIntel,
attackOrder, enemyBase)


      // Validasi target rush
      // jika invalid pindah ke kandidat base berikutnya
      ValidateOrRotateRushTarget()


      // Refill jika paint rendah dan dekat tower
      IF paint < SPLASHER_LOW_PAINT AND
DistanceTo(nearestTower) <= 2 THEN
        WithdrawPaintWithReserve(nearestTower)
      ENDIF


      // Attack pass 1
      bestAtk <- ArgMaxSplashAttack(centerPlus8Neighbors)

      IF bestAtk.score > 100 THEN
        Attack(bestAtk.loc)
      ENDIF


      // Movement greedy
      bestMove <- ArgMaxSplasherMove(8 directions)

      IF bestMove.exists THEN
        Move(bestMove.dir)
      ELSE
        RandomMove()
      ENDIF
```

```
      // Attack pass 2 (sesudah bergerak)
      bestAtk2 <-
ArgMaxSplashAttack(newCenterPlus8Neighbors)

      IF bestAtk2.score > 50 THEN
        Attack(bestAtk2.loc)
      ENDIF


      RobotReport(NULL)

    ENDIF



    IF unitType = MOPPER THEN

      ProcessMessages()

      nearestTower <- FindNearestAllyTower()
      hasEnemyPaintNearby <- ScanEnemyPaintNearby()


      bestAction <- ACT_NOTHING
      bestScore <- -INF


      // Kandidat 1: mop tile cat musuh
      EvaluateMopEnemyPaintTile()

      // Kandidat 2: mop swing cardinal
      EvaluateMopSwing()

      // Kandidat 3: transfer paint ke ally
      EvaluateTransferPaint(hasEnemyPaintNearby)

      // Kandidat 4: withdraw paint jika rendah
      EvaluateWithdrawPaintIfLow(nearestTower)
```

```
      // Kandidat 5: movement
      EvaluateMoveAllDirectionsMopper(nearestTower,
hasEnemyPaintNearby)


      Execute(bestAction)


      // Bonus action sesudah move
      BonusFollowupActionMopper()


      RobotReport(NULL)

    ENDIF


    Go()  // Akhiri turn

  ENDWHILE

END
```

2. Alternatif 2

```
BEGIN

  InitTaskStack()
  PushTask(EXPLORE)


  WHILE IsRunning DO

    UpdatePerception()
    ProcessMessages()


    // Rule transisi prioritas
```

```
IF paint sangat rendah THEN
  PushTask(REFILL)
ENDIF

IF enemyTower terlihat THEN
  PushTask(RUSH_TOWER)
ENDIF

IF ruin kosong dekat THEN
  PushTask(BUILD_TOWER)
ENDIF

IF enemyPaint banyak THEN
  PushTask(CLEAN_AREA)
ENDIF


task <- TopTask()


IF task = REFILL THEN

  IF canWithdraw THEN
    WithdrawPaint()
  ELSE
    MoveTo(nearestTower)
  ENDIF

  IF paint sudah aman THEN
    PopTask()
  ENDIF

ENDIF



IF task = RUSH_TOWER THEN

  IF canAttackTarget THEN
    AttackTarget()
```

```
    ELSE
      MoveTo(targetTower)
    ENDIF

    IF target invalid THEN
      PopTask()
    ENDIF

  ENDIF



  IF task = BUILD_TOWER THEN

    IF canCompleteTower THEN
      CompleteTower()

    ELSE IF canMarkPattern THEN
      MarkPattern()

    ELSE
      PaintPatternTile()
    ENDIF

    IF tower selesai dibangun THEN
      PopTask()
    ENDIF

  ENDIF



  IF task = CLEAN_AREA THEN

    IF unit = MOPPER THEN
      MopNearestEnemyPaint()
    ELSE
      PaintBestNonAllyTile()
    ENDIF
```

```
      IF area bersih THEN
        PopTask()
      ENDIF

    ENDIF



    IF task = EXPLORE THEN
      MoveToUnvisitedOrCenter()
    ENDIF


    IF noValidAction THEN
      RandomMove()
    ENDIF


    Go()

  ENDWHILE

END
```

3. Alternatif 3

```
BEGIN

  goals <- {
    CAPTURE_RUIN,
    CONTEST_RUIN,
    RUSH_TOWER,
    PUSH_FRONT,
    PAINT_AREA,
    EXPLORE
  }

  InitGlobalIntel()
```

```
  WHILE IsRunning DO

    ProcessMessagesWindow(last 5 rounds)
    UpdateGlobalIntel()


    FOR each g IN goals DO

      utility[g] <- 0

      utility[g] <- utility[g] + ScoreDistance(g)
      utility[g] <- utility[g] + ScoreUrgency(g, round)
      utility[g] <- utility[g] + ScoreResourceFit(g, paint,
chips)
      utility[g] <- utility[g] + ScoreThreat(g,
enemiesNearby)
      utility[g] <- utility[g] + ScoreRoleFit(g, unitType)

    ENDFOR


    bestGoal <- ArgMax(utility)


    IF unitType = SOLDIER THEN
      action <- ArgMax({
        complete,
        mark,
        paint,
        move,
        refill
      } constrained by bestGoal)
    ENDIF


    IF unitType = SPLASHER THEN
      action <- ArgMax({
        splashNow,
        splashAfterMove,
        moveToFront,
```

```
        refill
      } constrained by bestGoal)
    ENDIF


    IF unitType = MOPPER THEN
      action <- ArgMax({
        mop,
        mopSwing,
        transfer,
        move
      } constrained by bestGoal)
    ENDIF


    IF unitType = TOWER THEN
      action <- ArgMax({
        spawnSoldier,
        spawnSplasher,
        spawnMopper,
        attack
      } constrained by bestGoal)
    ENDIF


    Execute(action)

    ReportSummary(bestGoal, keyIntel)

    Go()

  ENDWHILE

END
```

4.2. Penjelasan Struktur Data, Fungsi, dan Prosedur

1. Run

Fungsi *Run* berperan sebagai pengendali utama yang menjalankan seluruh siklus logika bot pada setiap turn permainan. Pada awal eksekusi, prosedur ini melakukan inisialisasi lokasi spawn dan memperkirakan kemungkinan posisi basis musuh berdasarkan simetri peta yang digunakan sebagai referensi awal untuk eksplorasi maupun serangan. Selanjutnya fungsi ini menjalankan loop utama selama permainan masih berlangsung, membaca tipe unit yang sedang aktif, lalu memanggil prosedur perilaku yang sesuai seperti *RunSoldier*, *RunMopper*, *RunSplasher*, atau *RunTower* sehingga setiap unit dapat menjalankan logika yang berbeda sesuai perannya dalam tim.

```
PROCEDURE Run()
  InitSpawnAndPossibleEnemyBases()
  WHILE IsRunning DO
    turnCount <- turnCount + 1
    t <- GetType()
    IF t = SOLDIER THEN RunSoldier()
    ELSE IF t = MOPPER THEN RunMopper()
    ELSE IF t = SPLASHER THEN RunSplasher()
    ELSE RunTower()
    ENDIF
    Go()
  ENDWHILE
ENDPROCEDURE
```

2. RunTower

Fungsi *RunTower* mengatur perilaku tower dalam permainan yang meliputi produksi unit baru, serangan terhadap musuh dalam jangkauan, serta koordinasi informasi dengan unit lain. Pada setiap turn tower terlebih dahulu menjalankan *ProcessMessages* untuk memperbarui informasi mengenai kondisi di peta, kemudian melakukan serangan menggunakan *TowerAttackLowestHP* dengan memprioritaskan musuh yang memiliki HP paling rendah. Setelah itu tower mengevaluasi beberapa kandidat unit yang dapat diproduksi menggunakan fungsi *ScoreTowerSpawn* dan memilih unit dengan skor terbaik menggunakan pendekatan greedy sebelum membangunnya pada arah yang tersedia melalui *BuildRobot*.

```
PROCEDURE RunTower()
  ProcessMessages()
```

```
  TowerAttackLowestHP()

  bestScore <- -INF
  bestUnit <- NULL
  FOR each unit IN {SOLDIER, SPLASHER, MOPPER} DO
    score <- ScoreTowerSpawn(unit, round) + Random(0,29)
    IF score > bestScore THEN
      bestScore <- score
      bestUnit <- unit
    ENDIF
  ENDFOR

  dir <- FirstBuildableDirection(bestUnit)
  IF bestUnit != NULL AND dir != NULL THEN
    BuildRobot(bestUnit, dir)
  ENDIF

  TowerRelayAndCommand()
ENDPROCEDURE
```

3. RunSoldier

Fungsi *RunSoldier* mengatur perilaku unit Soldier yang memiliki peran utama dalam eksplorasi area, pembangunan tower, serta penguasaan wilayah peta. Unit ini terlebih dahulu memproses pesan menggunakan *ProcessMessages* lalu mencari lokasi penting seperti ruin terdekat melalui *FindNearestEmptyRuinOrIntel* dan tower sekutu terdekat melalui *FindNearestAllyTower*. Berdasarkan informasi tersebut Soldier mengevaluasi berbagai kemungkinan aksi seperti menyelesaikan pembangunan tower, memperbaiki pola cat, menandai pola tower, membangun pola resource, mengambil cat dari tower jika persediaan rendah, meningkatkan tower sekutu, atau bergerak ke arah tertentu melalui fungsi evaluasi yang tersedia, kemudian memilih aksi dengan skor tertinggi sebelum mengeksekusinya.

```
PROCEDURE RunSoldier()
  ProcessMessages()
  nearestRuin <- FindNearestEmptyRuinOrIntel()
  nearestTower <- FindNearestAllyTower()

  bestAction <- ACT_NOTHING
  bestScore <- -INF
```

```
  ConsiderCompleteTower()
  ConsiderPaintPatternTile()
  ConsiderMarkTowerPattern()
  ConsiderResourcePatternIfNoRuin()
  ConsiderWithdrawPaintIfLow(nearestTower)
  ConsiderUpgradeNearbyTower()
  ConsiderMove8Directions(nearestRuin, nearestTower,
attackOrder)

  Execute(bestAction)

  IF NeedBonusMoveToRuin() THEN MoveTo(nearestRuin)

  IF ActionReady() THEN
    TryCompleteTowerPriority()
    IfFailTryPaintPattern()
    IfFailTryPaintLocalTile()
  ENDIF

  RobotReport(nearestRuin)
ENDPROCEDURE
```

4. RunSplasher

Fungsi *RunSplasher* mengatur perilaku unit Splasher yang berfokus pada serangan area menggunakan kemampuan splash attack. Unit ini terlebih dahulu memproses pesan menggunakan *ProcessMessages* dan menentukan target serangan melalui *SelectRushTarget*. Setelah target ditentukan, unit akan mengevaluasi kemungkinan serangan pada area sekitar menggunakan *ArgMaxSplashAttack* yang memeriksa area 3×3 di sekitar posisi unit. Jika ditemukan serangan yang menguntungkan maka unit akan menyerang, kemudian bergerak menuju posisi yang lebih strategis menggunakan *ArgMaxSplasherMove* sebelum kembali mengevaluasi kemungkinan serangan tambahan.

```
PROCEDURE RunSplasher()
  ProcessMessages()
  nearestTower <- FindNearestAllyTower()
  rushTarget <- SelectRushTarget()
```

```
  ValidateOrRotateRushTarget()

  IF paint rendah AND dekatTower THEN
    WithdrawPaintWithReserve()
  ENDIF

  bestAtk <- ArgMaxSplashAttack(centerPlus8)
  IF bestAtk.score > 100 THEN Attack(bestAtk.loc)

  bestMove <- ArgMaxSplasherMove(rushTarget, nearestTower)
  IF bestMove.exists THEN Move(bestMove.dir)
  ELSE RandomMove()
  ENDIF

  bestAtk2 <- ArgMaxSplashAttack(newCenterPlus8)
  IF bestAtk2.score > 50 THEN Attack(bestAtk2.loc)

  RobotReport(NULL)
ENDPROCEDURE
```

5. RunMopper

Fungsi *RunMopper* mengatur perilaku unit Mopper yang berperan sebagai unit pendukung untuk membersihkan cat musuh dan membantu distribusi sumber daya cat kepada unit sekutu. Pada setiap turn unit memproses pesan melalui *ProcessMessages*, kemudian mendeteksi kondisi lingkungan seperti keberadaan cat musuh menggunakan *ScanEnemyPaint* dan mencari tower sekutu melalui *FindNearestAllyTower*. Berdasarkan kondisi tersebut Mopper mengevaluasi beberapa aksi seperti membersihkan cat musuh, melakukan *mop swing*, mentransfer cat ke unit sekutu, mengambil cat dari tower jika persediaan rendah, atau bergerak menuju posisi yang lebih strategis sebelum mengeksekusi aksi dengan skor terbaik.

```
PROCEDURE RunMopper()
  ProcessMessages()
  nearestTower <- FindNearestAllyTower()
  hasEnemyPaintNearby <- ScanEnemyPaint()

  bestAction <- ACT_NOTHING
  bestScore <- -INF
```

```
  ConsiderMopEnemyPaint()
  ConsiderMopSwing()
  ConsiderTransferPaintToAlly()
  ConsiderWithdrawIfLowPaint(nearestTower)
  ConsiderMove8Directions()

  Execute(bestAction)

  IF ActionReady() THEN
    TryMopEnemyPaintAgain()
    IfFailTryTransferPaintAgain()
  ENDIF

  RobotReport(NULL)
ENDPROCEDURE
```

6. ProcessMessages

Prosedur *ProcessMessages* bertugas membaca pesan yang dikirim oleh unit lain pada beberapa ronde sebelumnya agar bot dapat memperbarui informasi intel yang dimilikinya. Setiap pesan dibaca menggunakan *ReadMessages*, kemudian didekode melalui *Decode* untuk memperoleh tipe informasi, lokasi yang dilaporkan, dan data tambahan lainnya. Informasi tersebut kemudian disimpan melalui *UpdateLatestIntelByType* sehingga unit dapat menggunakan intel terbaru tersebut dalam proses pengambilan keputusan pada turn berikutnya.

```
PROCEDURE ProcessMessages()
  FOR r from currentRound-4 to currentRound DO
    msgs <- ReadMessages(r)
    FOR each msg IN msgs DO
      type, loc, extra <- Decode(msg)
      UpdateLatestIntelByType(type, loc, msg.round)
    ENDFOR
  ENDFOR
ENDPROCEDURE
```

7. RobotReport

Prosedur *RobotReport* digunakan oleh unit untuk mengirimkan laporan kepada tower sekutu mengenai informasi penting yang ditemukan di lapangan. Unit terlebih dahulu

mencari tower sekutu terdekat menggunakan *FindNearestAllyTower*, kemudian memeriksa keberadaan musuh melalui *SenseEnemies*. Jika ditemukan musuh maka informasi tersebut akan dikirim menggunakan *SendMessage* setelah dikodekan dengan *Encode*, sedangkan jika tidak ada musuh namun unit menemukan ruin kosong maka lokasi ruin tersebut juga akan dilaporkan agar unit lain dapat memanfaatkannya.

```
PROCEDURE RobotReport(nearestRuin)
  towerLoc <- FindNearestAllyTower()
  IF towerLoc = NULL THEN RETURN

  enemies <- SenseEnemies()
  IF enemies not empty THEN
    target <- PreferEnemyTowerElseFirstEnemy(enemies)
    SendMessage(towerLoc, Encode(enemyType, target.loc,
enemyCount))
    RETURN
  ENDIF

  IF nearestRuin != NULL THEN
    SendMessage(towerLoc, Encode(RUIN_FOUND, nearestRuin,
0))
  ENDIF
ENDPROCEDURE
```

8. Score Functions (heuristics)

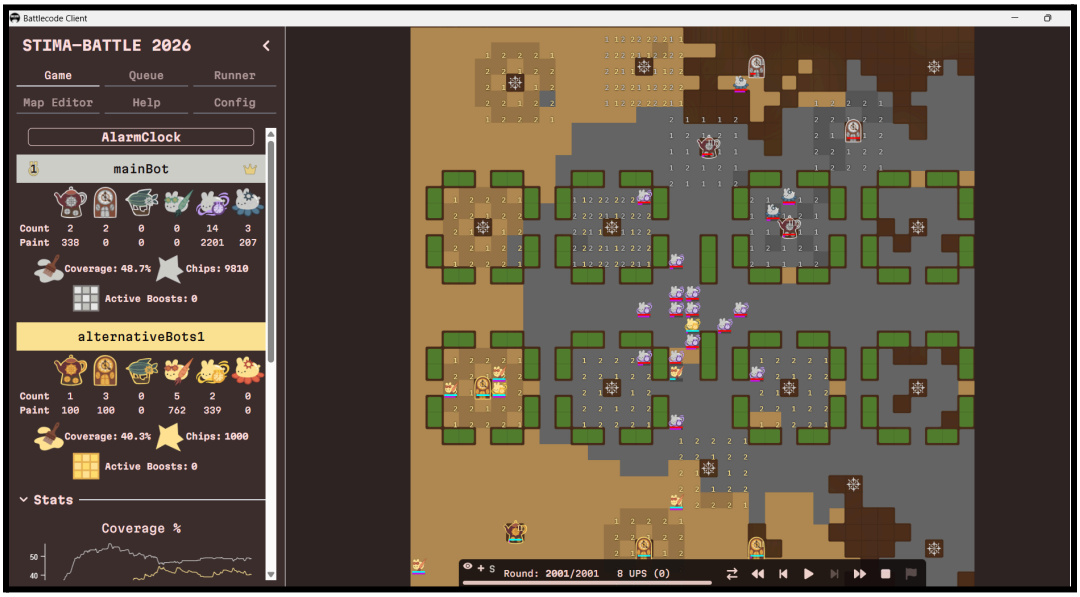
Fungsi *ScoreSoldierMove* digunakan untuk mengevaluasi kualitas suatu pergerakan yang dapat dilakukan oleh unit Soldier dengan menggunakan pendekatan heuristik. Fungsi ini menghitung skor berdasarkan berbagai faktor lingkungan seperti kedekatan dengan ruin, target serangan, pusat peta, serta kondisi tile tujuan. Selain itu fungsi ini juga mempertimbangkan kondisi sumber daya cat yang dimiliki unit sehingga pergerakan menuju tower sekutu dapat diprioritaskan ketika persediaan cat rendah.

```
FUNCTION ScoreSoldierMove(dir, nearestRuin, nearestTower,
paint, attackTarget) RETURNS integer
  score <- EXPLORE_BASE

  IF nearestRuin ada AND dir mendekat ke ruin THEN score
<- score + BONUS_RUIN
  ELSE IF attackTarget ada AND dir mendekat target THEN
```

```
score <- score + BONUS_ATTACK
  ELSE IF dir mendekat center THEN score <- score +
BONUS_CENTER
  ENDIF

  IF targetTile enemyPaint THEN score <- score +
PENALTY_ENEMY_TILE
  IF targetTile empty THEN score <- score +
BONUS_EMPTY_TILE
  IF targetTile wall THEN RETURN -INF

  IF paint rendah AND nearestTower ada AND dir mendekat
tower THEN
    score <- score + LOW_PAINT_PULL_BONUS
  ENDIF

  RETURN score
ENDFUNCTION
```

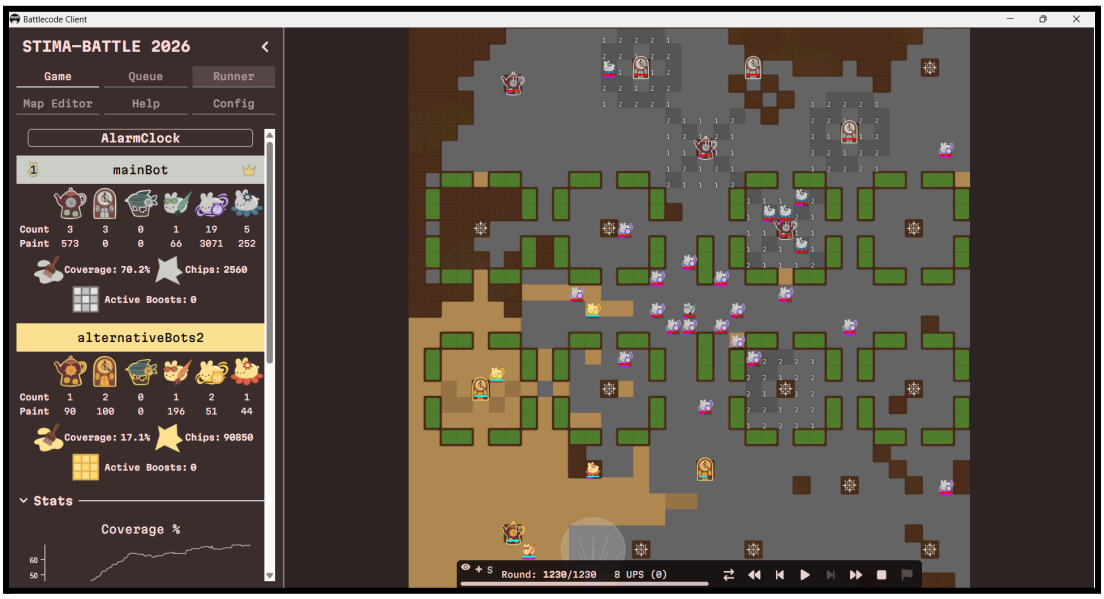
4.3. Pengujian Greedy

1. Pengujian 1 (Main Bot vs Alternative Bot 1) – AlarmClock

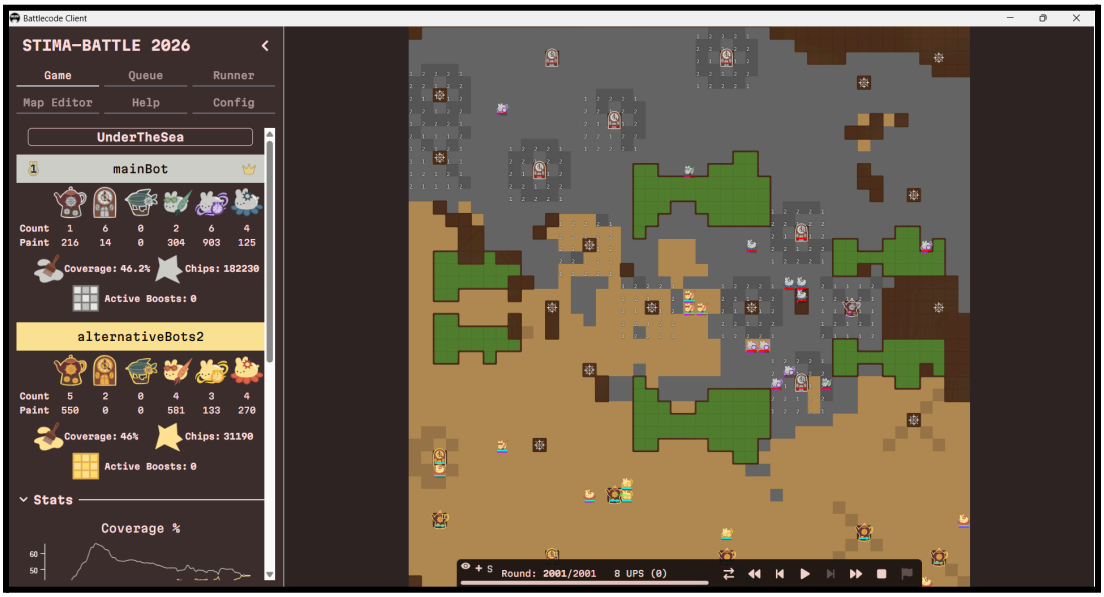


2. Pengujian 2 (Main Bot vs Alternative Bot 1) – Leaf

3. Pengujian 3 (Main Bot vs Alternative Bot 2) – AlarmClock



4. Pengujian 4 (Main Bot vs Alternative Bot 2) – UnderTheSea



4.4. Analisis Hasil Pengujian

1. Pengujian 1 (Main Bot vs Alternative Bot 1) – AlarmClock

   Pada Pengujian 1 di *map AlarmClock*, *Main Bot* berhasil mengalahkan *Alternatif Bot 1* dengan penguasaan wilayah sebesar 48,7%, di mana memunculkan sebuah kejadian unik berupa anomali ekonomi. Alternatif 2 yang difokuskan sepenuhnya secara greedy untuk membangun Money Tower justru berakhir dengan kemiskinan (1.000 Chips) akibat unitnya yang terus terbunuh saat melakukan rush buta, berbanding terbalik dengan Main Bot yang sukses menimbun 9.810 Chips berkat evaluasi greedy *mikro-manajemen unit* yang tangguh. Dari hasil tersebut, strategi greedy pada *Main Bot* terbukti berhasil mencapai nilai optimal dalam hal efisiensi pertempuran lokal dan manajemen logistik cat secara mandiri pada setiap gilirannya. Namun, strategi greedy ini juga terbukti gagal mendapatkan nilai optimal dalam aspek pergerakan global. Pada kondisi lingkungan AlarmClock yang didominasi oleh dinding dan sekat ruangan tertutup, tidak adanya algoritma *pathfinding* menyebabkan beberapa *unit Main Bot* membuang gilirannya akibat terjebak berjalan maju membentur sudut dinding.

2. Pengujian 2 (Main Bot vs Alternative Bot 1) – Leaf

   Pada Pengujian 2 di *map Leaf*, Main Bot meraih kemenangan telak atas *Alternatif Bot 1* dengan penguasaan wilayah 70% pada putaran ke-1379. Kejadian unik yang mencolok adalah hancurnya seluruh armada Alternatif 2, meskipun mereka menimbun 6.860 *Chips* yang terbuang sia-sia, berbanding terbalik dengan *Main Bot* yang sukses mengekspansi 90 unit *Soldier*. Dari hasil ini, strategi *greedy* pada *Main Bot* terbukti berhasil mencapai

nilai sangat optimal. Topografi peta *Leaf* yang sangat terbuka menutupi kelemahan bawaan algoritma *greedy* (rentan terjebak di jalan buntu atau local optima), sehingga setiap keputusan jarak dekatnya selaras sempurna dengan dominasi global. Sebaliknya, area tanpa sekat ini justru mengeksploitasi kelemahan Alternatif 2, instruksi *rush* (serangan) buta mereka membuat unit yang maju sendirian langsung dieliminasi secara instan oleh kerumunan *Main Bot* sebelum sisa Chips tersebut sempat digunakan menara untuk memproduksi pasukan baru.

3. Pengujian 3 (Main Bot vs Alternative Bot 2) – AlarmClock

   Pada Pengujian 3 di *map AlarmClock*, Main Bot meraih kemenangan telak atas *Alternative Bot* 2 pada putaran ke-1230 dengan dominasi wilayah 70,2%. Kejadian paling unik adalah anomali fatal pada kubu lawan yang menimbun sumber daya hingga 90.850 *Chips* tanpa membelanjakannya, sehingga armada tempur mereka nyaris lumpuh. Dari hasil ini, strategi greedy Main Bot terbukti sangat optimal dalam perputaran ekonomi, secara rakus mengonversi *Chips* menjadi 25 pasukan tempur untuk mengunci musuh. Sebaliknya, Alternative Bot 2 gagal mencapai nilai optimal, meskipun dibekali sistem komunikasi canggih, fungsi produksi dan pergerakannya diduga terjebak pada local optima akibat rute yang terhalang sekat dinding peta. Hal ini menegaskan bahwa pada lingkungan berlorong tertutup, efisiensi greedy murni yang berfokus pada *mikro-manajemen* lokal (Main Bot) jauh lebih efektif dan stabil daripada sinkronisasi taktis global yang macet akibat rintangan spasial.

4. Pengujian 4 (Main Bot vs Alternative Bot 2) – UnderTheSea

   Pada Pengujian 4 di *map UnderTheSea*, *Main Bot* meraih kemenangan sangat tipis atas *Alternative Bot* 2 pada putaran batas akhir 2001 (penguasaan 46,2% vs 46%). Kejadian paling unik adalah anomali penimbunan kekayaan ekstrem oleh Main Bot hingga 182.230 *Chips* yang gagal dimanfaatkan untuk agresi teritorial. Dari hasil ini, strategi greedy pada Main Bot terbukti gagal mencapai nilai optimal dalam utilisasi sumber daya. Fokus statisnya yang terus mendirikan *Money Tower* membuatnya lumpuh dan tidak mampu beradaptasi ke orientasi cat (*Paint Tower*) di fase late game. Sebaliknya, *Alternative Bot 2* nyaris membalikkan keadaan berkat heuristik adaptifnya yang sukses mendirikan 5 *Paint Tower*. Hal ini menegaskan kelemahan utama algoritma *greedy* statis, ketidakmampuan mengubah fungsi objektif seiring berjalannya fase permainan membuatnya rentan membuang potensi sumber daya (local optima ekonomi) dan nyaris kalah di peta berstruktur kompleks.

V. Kesimpulan dan Saran

5.1. Kesimpulan

Melalui Tugas Besar I Strategi Algoritma ini, kami berhasil membangun bot otonom untuk permainan Battlecode 2025 menggunakan algoritma greedy. Program final kami mengimplementasikan Alternatif 1 sebagai strategi utama karena efisiensi komputasi O(1) mutlaknya melalui sistem skor terpadu yang mengevaluasi probabilitas opsi secara paralel. Strategi ini sangat tangguh dalam mikro-manajemen unit dan ekspansi ekonomi, namun memiliki kelemahan bawaan yaitu rentan terjebak pada local optima saat harus menavigasi rintangan dinding yang kompleks.

5.2. Saran

Bot ini dapat dikembangkan lebih lanjut dengan mengintegrasikan algoritma pencarian jalur lanjutan (*pathfinding*) agar pergerakan unit tidak membuang giliran akibat terjebak di jalan buntu atau sudut ruangan. Selain itu, heuristik statis pada pembangunan menara dapat dibuat lebih dinamis agar bot mampu beradaptasi mengubah fokus ke Paint Tower saat memasuki fase late game, sehingga timbunan sumber daya yang berlebih dapat dimaksimalkan untuk dominasi wilayah.

VI. Lampiran

Link Repository Github : NazwanSM/Tubes1_Probe: Tugas Besar 1 - Strategi Algoritma
Link Video Youtube : https://youtu.be/JKOATcV6FLE?si=8OXpcp6mhPZ9V01w

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Adam Mirza

Nazwan Siddqi Muttaqin

Matthew Sebastian Kurniawan

VII. Daftar Pustaka

Ottesen, J., Bank, A., dan Voynovich, M. 2025. The Kragle - Battlecode 25 Postmortem [PDF]. Tersedia pada: https://github.com/justinottesen/battlecode25 (diakses 15 Maret 2026).

MIT Battlecode. 2026. Battlecode. Tersedia pada: https://battlecode.org/ (diakses 9 Maret 2026).

Battlecode Java API. 2026. *Battlecode25 Java Documentation – battlecode.common Package*. Tersedia pada:https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html (diakses 9 Maret 2026).

MIT Battlecode. 2025. Battlecode 2025 Pathfinding [Lecture 5]. Video YouTube. Tersedia pada: https://www.youtube.com/watch?v=Mqk50BQH3oQ (diakses 11 Maret 2026).